ZeniaHR

Employee Mobile App

API Integration & Developer Handover

Flutter Client ↔ ZeniaHR Website (Single Source of Truth)

Backend API base • `/api/app` • v1

Document type: Backend API Audit + Flutter Integration Specification

Prepared for: Flutter Mobile Application Developer

Scope: Employee-only mobile application (Android / iOS)

Status: Backend audited against live source code — reuse-first

Table of Contents

> **How to read this document.** Every endpoint, table, and field named here was verified against the live backend source (`backend/src/app/*` , `backend/prisma/schema.prisma`). Endpoints marked **LIVE** exist today. Endpoints marked **TO BUILD** are

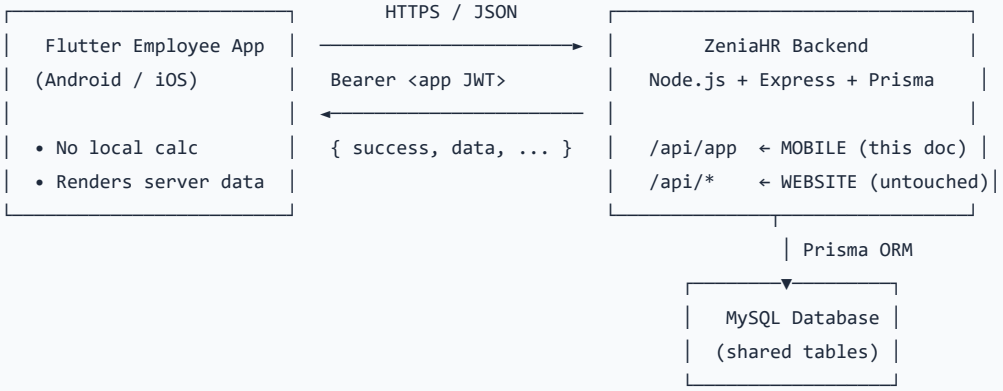
> recommended additions that reuse an existing database table — **no new business logic and no duplicate endpoints.**

01 System Overview

The ZeniaHR website is the single source of truth. The Flutter app is a thin client that only reads and writes through the dedicated mobile API namespace `/api/app`. The app never computes salary, attendance, payroll, leave, or loan values locally — it renders what the server returns.

/api/app	28	13	1
ISOLATED MOBILE NAMESPACE	ENDPOINTS LIVE TODAY	ENDPOINTS TO ADD (REUSE TABLES)	BLOCKING AUTH GAP (G1)

Architecture at a glance



Key design principle — separation without duplication. The mobile API lives entirely under `/api/app` and is mounted independently in `server.js` (`app.use('/api/app', require('./src/app/routes'))`). It has its own auth middleware, its own response envelope, and its own controllers — but it reads and writes the **same database tables** as the website (`Employee` , `TemporaryEmployee` , `Attendance` , `LeaveRequest` , `Payroll` , `Loan` , `Notification` , …). Changing the website does not change the app, and vice-versa, yet both always see the same data.

Response envelope (every endpoint)

```
{
  "success": true,
  "message": "Human-readable message",
  "data": { ... },          // null on failure
  "errors": [],             // [{ code, message, field? }] on failure
  "timestamp": "2026-07-15T09:12:44.301Z"
}
```

Technology & conventions

Concern	Convention
Base URL	`https://<host>/api/app`
Content type	`application/json` (files sent as base64 data URLs, see §20)
Auth header	`Authorization: Bearer <accessToken>`

Identity	App tokens are keyed to a `TemporaryEmployee` (onboarding identity), or — once G1 is applied — an `Employee`.
Dates	Stored as `YYYY-MM-DD` strings; times as `HH:mm` strings.
Money	Numeric (float), computed server-side only.

02 Authentication Flow

Mobile authentication is **mobile-number + OTP**, completely separate from the website's email/password login. Only employees can obtain an app token.

OTP behaviour (current phase)

Setting	Value / env	Behaviour
OTP mode	`OTP_MODE` (default `development`)	**Dev:** any correct-length digit string is accepted. No SMS is sent. **Production:** wire real SMS verification in `appAuth.verifyOtpSession()`.
OTP length	`OTP_LENGTH` (default 4)	Client should render N input boxes from `otpLength`.
OTP TTL	`OTP_EXPIRY_SECONDS` (default 300)	Session expires after 5 min → request a new OTP.
Attempts	5	After 5 wrong attempts the session is destroyed (429).
Dev hint	`devOtp`	In dev, the login response includes `devOtp` so QA can auto-fill.

> **Future SMS integration is a one-file change.** The OTP session store and verify function already exist; a production SMS provider only needs to (a) send the generated code on login and (b) compare it in `verifyOtpSession`. No client change is required — the flow and payloads stay identical.

Token lifecycle

- **Access token** — `kind:"app"`, expiry `APP_ACCESS_TOKEN_EXPIRY` (default 12h). Sent on every request.
- **Refresh token** — `kind:"app-refresh"`, expiry `APP_REFRESH_TOKEN_EXPIRY` (default 30d). Used only at `/auth/refresh`.
- On `401 TOKEN_INVALID`, the app calls `/auth/refresh`; if that also fails, force re-login.

03 Employee Login Flow

The app is **employees only**. Super Admin, Company Head, HR, and Branch Admin are website `User` accounts — they are structurally unable to obtain an app token because app tokens are keyed to employee identities, not `User` rows.

Who can log in

Role	Backing table	Can log into app?	Why
Employee (permanent)	`Employee`	**YES***	*After gap G1 is applied — see the callout below.
Onboarding / Temporary Employee	`TemporaryEmployee`	**YES**	Native identity of the app today.
Super Admin / Company Head / HR / Branch Admin	`User`	**NO**	App tokens are never issued for `User` rows.

> **Gap G1 (must fix for full Phase-3 compliance).** Today `authController.findTempByMobile()` resolves the mobile number **only** against `TemporaryEmployee.mobile`. A permanent employee whose number lives only in `Employee.phone` (and was not created through the temp→convert flow) will get `404 EMPLOYEE_NOT_FOUND`. **Fix:** add an `Employee.phone` fallback in login; when matched, issue an `employee`-kind token and load the `Employee` directly in `appProtect`. This is the only blocking change; everything downstream already works once the identity resolves.

Client routing after verify-otp

```
verify-otp success
      │
      ├─ approvalStatus == "Approved"        ──►  DASHBOARD
      ├─ approvalStatus == "Pending Approval"──►  "Waiting for Approval" screen
      ├─ approvalStatus == "Rejected"        ──►  Rejection screen (show remarks, allow edit + resubmit)
      ├─ approvalStatus == "Changes Requested"──► Edit screen (show note, allow resubmit)
      └─ else (Draft / incomplete)           ──►  ONBOARDING at `currentStep`
```

04 Onboarding Flow

First-time employees complete a **7-step** registration. Each step saves a draft to the shared `TemporaryEmployee` record, recomputes completion %, and advances the step pointer. Nothing is lost on app restart; editing is allowed any time until approval.

The 7 steps

#	Step	Endpoint	Completion rule (server-side)
1	Personal	POST `/profile/personal`	name + (dob/gender/firstName)
2	Address	POST `/profile/address`	present address present
3	Family / Nominee	POST `/profile/family`	nominee OR emergency contact OR father/spouse
4	Bank	POST `/profile/bank`	accountNumber + ifsc
5	Education	POST `/profile/education`	≥1 education entry
6	Experience	POST `/profile/experience`	≥1 experience entry
7	Documents	POST `/profile/documents`	photo + aadhaarDoc + panDoc + bankProof

Supporting endpoints

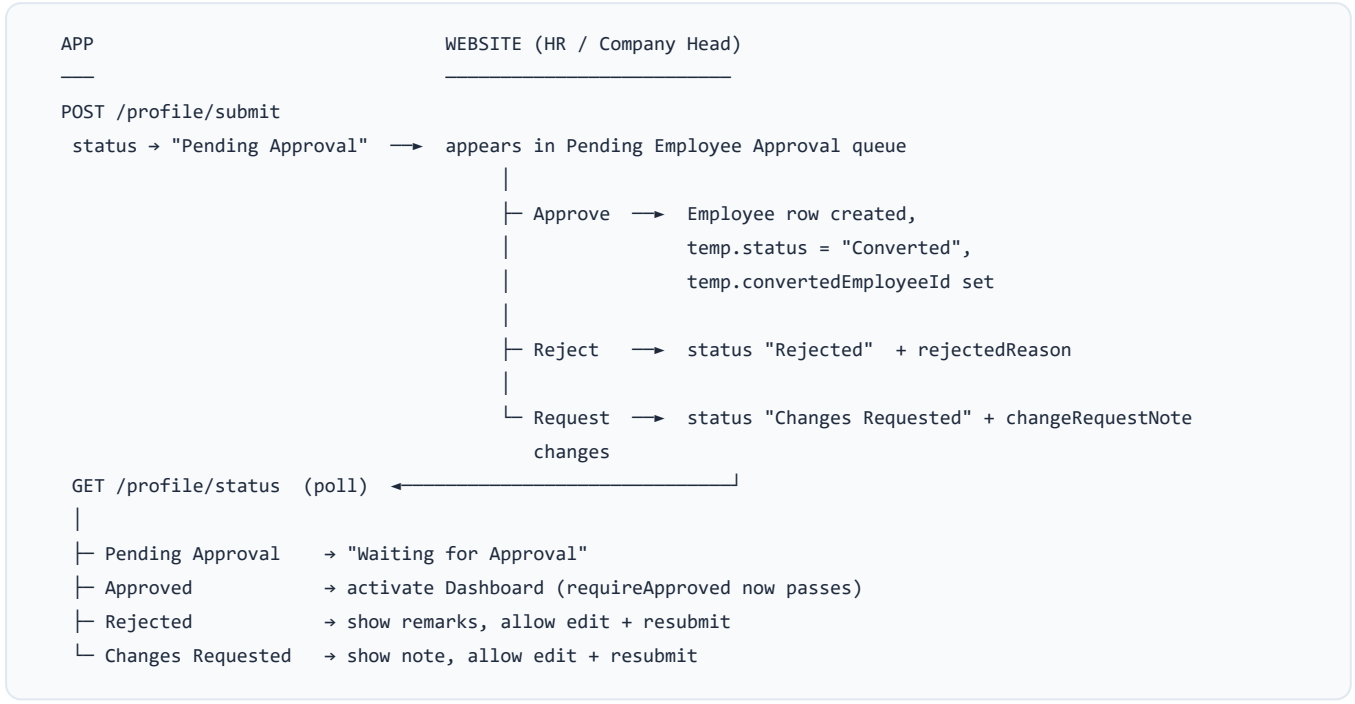
Endpoint	Purpose
POST `/profile/document`	Upload a single document `{ type, name, dataUrl }`. `type` ∈ photo/aadhaar/pan/passbook/signature/degree/education/experience/other.
GET `/profile/progress`	`{ currentStep, completedSteps[], completionPercentage, totalSteps, canSubmit, approvalStatus }`
PUT `/profile/update`	Patch any draft fields (before approval); limited self-service fields (after approval).
POST `/profile/submit`	Validate mandatory fields+docs → set status `Pending Approval`.
GET `/profile/status`	Poll approval status + remarks.

Submission gate (mirrors HR's mandatory checks)

Mandatory fields: `name, mobile, presentAddress, aadhaar, pan, accountNumber` + branch. Mandatory docs: `photo, aadhaarDoc, panDoc, bankProof`. If any are missing, `/profile/submit` returns `422 VALIDATION_FAILED` with a per-field `errors[]` list — render these inline against each field.

05 Approval Flow

Submission places the record into the website's existing **Pending Employee Approval** queue. HR / Company Head review it on the website; the app polls status and reacts.



Status vocabulary mapping

TemporaryEmployee.status (DB)	App approvalStatus	App behaviour
Pending Profile / Partially Completed	Draft	Continue onboarding
Pending Approval	Pending Approval	Waiting screen
Changes Requested	Changes Requested	Edit + resubmit (show note)
Rejected	Rejected	Edit + resubmit (show reason)
Converted	Approved	Dashboard unlocked

Automatic dashboard activation. Once HR approves, `temp.status="Converted"` and a real `Employee` is linked. The `requireApproved` middleware then passes and all dashboard endpoints become available with no further app action — the app just needs to re-check `/auth/session`.

06 Dashboard Flow

The dashboard is available **only after approval** (`requireApproved`). All figures are live from the website — the app performs no calculation.

`GET` `/api/app/dashboard` — current fields

```
{ "employeeId", "name", "designation", "department",
  "company", "branch",
  "todayAttendance": { "status", "clockIn", "clockOut" },
  "pendingLeaveRequests", "latestPayrollNet" }
```

> **Gap G13 — dashboard enrichment.** The Phase-6 spec asks for a richer home screen. Every missing field already exists in a table and can be folded into this one endpoint (recommended) or fetched via the dedicated endpoints in the following sections. Mapping:

Requested tile	Source (existing)	Status
Employee name / photo / code	`Employee.name / photoUpload / employeeId`	**add photo + code**
Designation / Department / Branch / Company	`Employee` + `Branch` + `Company`	**present**
Attendance today	`Attendance` (today row)	**present**
Monthly attendance (working/present/absent)	`AttendanceSummary`	**add (§07)**
Leave balance	`LeaveBalance` (cl/pl/sl)	**add (§08)**
Upcoming holidays	`CommunicationHoliday`	**stub → build (§07)**
Shift details	`Shift` via `Employee.shiftId`	**add (§07)**
CTC / Monthly / Net salary	`Employee.salary` (monthly; CTC = ×12), `Payroll.netSalary`	**add CTC + monthly**
Latest payslip	`Payroll` (latest)	**net present; add slip ref**
Notifications / Announcements	`Notification` / `CommunicationAnnouncement`	**present**
Loan / Advance summary	`Loan`	**add (§09/§10)**
PF / ESIC	`Employee.pfNumber / esiNumber / uan`	**add**

> **CTC rule (important).** `Employee.salary` is a **monthly** figure. Annual CTC = `salary × 12` . Do not display `salary` as annual, and do not divide by 12 anywhere in the app — the server sends final values.

07 Attendance APIs

Employees can **view** attendance; they can never edit it. Attendance originates on the website (biometric import, manual, or device sync).

`GET` `/api/app/attendance` LIVE

Returns the last 60 rows for the logged-in employee + a summary.

```
{ "summary": { "totalRecords", "presentDays" },
  "records": [{ "date", "status", "clockIn", "clockOut", "hoursWorked" }] }
```

> **Recommended params (Phase 15):** add `?month=&year=&page=&limit=`. The underlying `Attendance` table has `date` (YYYY-MM-DD), `status`, `clockIn/Out`, `hoursWorked`, `leaveType`, `shift`.

To build — reuse existing tables

Endpoint	Reuse	Returns
`GET` `/api/app/attendance/monthly` **TO BUILD**	`AttendanceSummary`	workingDays, presentDays, absentDays, weeklyOff, holidayDays, payableDays
`GET` `/api/app/overtime` **TO BUILD** (G6)	`Overtime` by employeeId	date, inTime, outTime, otHours, type, status
`GET` `/api/app/shift` **TO BUILD** (G5)	`Shift` via `Employee.shiftId`	name, start, end, grace, breakTime
`GET` `/api/app/holiday` **STUB → BUILD** (G4)	`CommunicationHoliday`	name, date, category, upcoming (filter companyId + branch + date ≥ today)

Late marks / early exit are derivable from `Attendance.status`, `clockIn/Out` vs the employee's `Shift.start/end` + `grace` — compute server-side in the monthly endpoint, never on the client.

08 Leave APIs

Employees apply for leave, view balance and history, and cancel their own pending requests. Approval is controlled on the website.

Endpoint	Status	Notes
GET `/api/app/leave`	LIVE	Own request history (latest 50).
POST `/api/app/leave/apply`	LIVE	`{ leaveType, fromDate, toDate, days, reason }` → creates `LeaveRequest` status `Pending`. Server sets companyId, employeeId, name, department.
GET `/api/app/leave/balance` TO BUILD (G2)	reuse	`LeaveBalance`: clBalance/plBalance/slBalance + used + carryForward for the year.
DELETE `/api/app/leave/:id` TO BUILD (G3)	reuse	Cancel own request — **only** when `status="Pending"` AND `employeeId` matches. Mirror the website's delete/status guard.

Ownership guard for cancel. The cancel endpoint must first load the `LeaveRequest` and confirm `row.employeeId === req.appCtx.employee.id` and `row.status === "Pending"` before deleting. Never trust an id from the client without this check (see §25).

Apply request body

```
{ "leaveType": "Casual Leave",
  "fromDate": "2026-07-20",
  "toDate":   "2026-07-21",
  "days":     2,
  "reason":   "Family function" }
```

09 Loan APIs

Employees apply for loans, attach documents, and track status. All EMI/interest computation is server-side (the website's Loan engine). The website approves.

> **Reuse note.** The website already exposes an employee self-service loan view at `GET /api/loans/portal/me` and a full Loan engine (`loanController` , `Loan` table with EMI/interest fields). The app endpoints below wrap the same table and logic under `/api/app` — no new EMI math.

Endpoint	Status	Notes
GET `/api/app/loans` **TO BUILD** (G7)	reuse	Own loans: loanNumber, loanTypeName, principalAmount, emiAmount, tenureMonths, status, outstanding.
POST `/api/app/loans` **TO BUILD** (G7)	reuse	Apply: `{ loanTypeId, principalAmount, tenureMonths, purpose, attachments[] }` → status `Pending Approval` .
GET `/api/app/loans/:id` **TO BUILD**	reuse	Detail + installment schedule (`LoanInstallment`). Ownership-guarded.
GET `/api/app/loans/types` **TO BUILD**	reuse	Company loan types (`LoanType`) for the apply form.

Loan workflow (server-owned)

```
Draft → Pending Approval → Approved → Disbursed → Running → Completed → Closed
                         └──────→ Rejected (rejectionReason)
```

The app displays `status` and, once `Running` , the EMI deducts automatically inside payroll (the website's loan→payroll spine). The app never computes an EMI.

10 Salary Advance APIs

A salary advance is modelled as a `Loan` of an "advance" loan type (the `Loan.purpose` field is explicitly noted for advances). It reuses the same table, workflow, and approval as loans.

Endpoint	Status	Notes
GET `/api/app/advances` **TO BUILD** (G8)	**reuse**	Own advances (Loan rows of advance type).
POST `/api/app/advances` **TO BUILD** (G8)	**reuse**	Apply: `{ amount, reason, month, year }` → creates a Loan of advance type, status `Pending Approval`.

> **Implementation tip.** Rather than a new controller, the advances endpoints can filter/create `Loan` rows where the loan type is flagged as an advance. This keeps one approval queue and one deduction path on the website. Confirm the exact advance loan-type key with the backend team before wiring.

11 Payroll APIs

Employees view their own payroll history. All amounts are computed and finalised on the website's payroll engine.

`GET` `/api/app/payroll` **LIVE**

```
{ "payslips": [{
    "id", "month", "year", "basicSalary", "allowances",
    "deductions", "netSalary", "paymentStatus", "payslipGenerated"
  }] }
```

Returns the latest 24 payroll rows for the employee, ordered newest-first.

> **Recommended params:** `?month=&year=` to fetch a single period. The `Payroll` table carries per-component breakup (basic, allowances, deductions, loanDeduction, netSalary, paymentStatus, payslipGenerated) plus snapshot columns for payable/working days used in proration.

Amounts the app must show verbatim

Field	Meaning
`basicSalary`	Prorated basic for the period
`allowances`	Sum of earnings components
`deductions`	Statutory + loan/advance deductions
`netSalary`	Final take-home (server-computed)
`paymentStatus`	Unpaid / Paid
`payslipGenerated`	Whether a slip PDF is available (§12)

12 Payslip APIs

The payslip is a rendered document (branded) generated by the website. The app should download/display it rather than reconstruct it.

Endpoint	Status	Notes
GET `/api/app/payroll`	LIVE	List with `payslipGenerated` flag (§11).
GET `/api/app/payroll/:id/payslip` **TO BUILD** (G12)	reuse	Return the branded slip. Reuse the website's slip renderer (the payroll module already renders + emails slips via `/:id/email-slip`). Serve as PDF (base64 or URL) or structured JSON for native rendering. Ownership-guarded to the employee's own payroll id.

> **Two viable approaches.** (1) Server returns a PDF (base64 data URL or short-lived link) — the app opens it in a PDF viewer; simplest and pixel-consistent with the website. (2) Server returns structured slip JSON and the app renders a native layout — better UX offline, more work. Recommendation: start with (1) reusing the existing renderer, add (2) later if needed.

13 Notification APIs

Live notifications and company announcements. Read is live; mark-as-read should be added to mirror the website.

Endpoint	Status	Notes
GET `/api/app/notifications`	LIVE	Latest 50 for the employee's company/branch. Fields: id, type, title, message, read, timestamp.
PUT `/api/app/notifications/:id/read` TO BUILD (G10)	reuse	Mark one read. Website already has `PUT /:id`.
PUT `/api/app/notifications/read-all` TO BUILD (G10)	reuse	Mark all read. Website already has `/read-all`.

Notification types the app should render

Announcements, Leave Approval, Loan Approval, Advance Approval, Payroll Generated, Payslip Ready, Document Rejected, Attendance Alerts, Company Notices. These arrive via the `type` field on `Notification`; map each to an icon/route in the app. Company-wide announcements also live in `CommunicationAnnouncement`.

> **Scoping.** Notifications are filtered by `companyId` and `branchId` (and, where set, `userId`). An employee only ever receives their company/branch notifications — never another tenant's.

14 Document Upload APIs

During onboarding, documents attach to the `TemporaryEmployee` record; after approval they belong to the `Employee`.

Endpoint	Status	Notes
POST `/api/app/profile/document`	LIVE	Onboarding single upload `{ type, name, dataUrl }`.
POST `/api/app/profile/documents`	LIVE	Onboarding bulk save.
GET `/api/app/profile/documents`	LIVE	Read (post-approval) — photo + documents JSON.
POST `/api/app/documents` **TO BUILD** (G11)	reuse	Post-approval upload of new docs to `Employee.documents` / the `Document` table.

Supported document types

Aadhaar, PAN, Photo, Signature, Bank Passbook, Cancelled Cheque, Driving Licence, Passport, and any company documents. Each maps to a storage key (e.g. `aadhaar→aadhaarDoc`, `pan→panDoc`, `passbook→bankProof`). See §20 for file format and size rules.

15 Profile APIs

Identity, onboarding draft, and the approved employee profile.

Endpoint	Status	Notes
GET `/api/app/auth/me`	LIVE	Onboarding identity (tempEmployeeId, name, mobile, approvalStatus, currentStep, %).
GET `/api/app/profile`	LIVE	Full approved employee profile (see field list below).
PUT `/api/app/profile/update`	LIVE	Draft edits before approval; limited self-service after approval (`email, phone, presentAddress, permanentAddress, emergencyContact`).

Approved profile payload (`GET /api/app/profile`)

```
{ "employeeId", "name", "firstName", "lastName", "email", "mobile",
  "department", "designation",
  "company": { id, name }, "branch": { id, name },
  "joinDate", "dob", "gender", "bloodGroup",
  "aadhaar", "pan", "bankName", "accountNumber", "ifsc",
  "presentAddress", "permanentAddress", "profileImage" }
```

> **Company/Branch info (G9).** A dedicated `GET /api/app/company` can return company name, logo, address, and the employee's branch details from `Company` + `Branch` for a "Company" screen.

16 API Request Examples

Login

```
POST /api/app/auth/login
Content-Type: application/json

{ "mobile": "9876543210" }
```

Verify OTP

```
POST /api/app/auth/verify-otp

{ "sessionId": "a1b2c3...", "otp": "1111" }
```

Authenticated request (all protected endpoints)

```
GET /api/app/dashboard
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Save onboarding step (personal)

```
POST /api/app/profile/personal
Authorization: Bearer <token>

{ "firstName": "Ravi", "lastName": "Kumar",
  "dob": "1996-04-12", "gender": "Male",
  "maritalStatus": "Single", "bloodGroup": "O+" }
```

Upload a single document

```
POST /api/app/profile/document
Authorization: Bearer <token>

{ "type": "aadhaar",
  "name": "aadhaar-front.jpg",
  "dataUrl": "..." }
```

Apply for leave

```
POST /api/app/leave/apply
Authorization: Bearer <token>

{ "leaveType": "Casual Leave", "fromDate": "2026-07-20",
  "toDate": "2026-07-21", "days": 2, "reason": "Family function" }
```

Submit for approval

```
POST /api/app/profile/submit
Authorization: Bearer <token>
(no body)
```

17 API Response Examples

Login success (dev mode)


```
{ "success": true, "message": "OTP generated.",
  "data": { "sessionId": "a1b2c3...", "otpRequired": true,
    "otpLength": 4, "expiresIn": 300,
    "otpMode": "development", "devOtp": "1111" },
  "errors": [], "timestamp": "..." }
```


Login — employee not found


```
{ "success": false, "message": "Employee not found.",
  "data": null,
  "errors": [{ "code": "EMPLOYEE_NOT_FOUND", "message": "Employee not found." }],
  "timestamp": "..." }
```


Verify OTP success


```
{ "success": true, "message": "OTP verified.",
  "data": {
    "accessToken": "eyJ...", "refreshToken": "eyJ...",
    "tokenType": "Bearer", "expiresIn": 43200, "refreshExpiresIn": 2592000,
    "registrationCompleted": false, "currentStep": 3,
    "approvalStatus": "Draft" }, "errors": [] }
```


Onboarding step saved


```
{ "success": true, "message": "Personal information saved.",
  "data": { "saved": true, "currentStep": 2,
    "completedSteps": [1], "completionPercentage": 14,
    "totalSteps": 7, "canSubmit": false, "approvalStatus": "Draft" } }
```


Validation failure (submit)


```
{ "success": false, "message": "Please complete all required fields and documents before submitting.",
  "data": null,
  "errors": [
    { "code": "MISSING_FIELD", "field": "pan", "message": "PAN Number is required." },
    { "code": "MISSING_DOCUMENT", "field": "bankProof", "message": "Bank Passbook / Proof is required." }
  ] }
```

18 Error Codes

All errors follow the standard envelope with a machine-readable `code` in `errors[]`. Map each to a user-facing message and a recovery action in the app.

HTTP	code	Meaning / recovery
400	`MOBILE_REQUIRED`	Mobile missing → validate before submit.
404	`EMPLOYEE_NOT_FOUND`	Mobile not registered → show "Employee not found."
400	`INVALID_SESSION`	OTP session unknown → request new OTP.
400	`OTP_EXPIRED`	OTP window passed → request new OTP.
400	`INVALID_OTP`	Wrong code → let user retry (attempts counted).
429	`TOO_MANY_ATTEMPTS`	5 wrong attempts → force new OTP request.
400	`REFRESH_TOKEN_REQUIRED`	No refresh token supplied.
401	`REFRESH_TOKEN_INVALID`	Refresh expired/invalid → force re-login.
401	`NO_TOKEN`	Missing Authorization header.
401	`TOKEN_INVALID`	Access token bad/expired → try refresh, else re-login.
404	`ACCOUNT_NOT_FOUND`	Identity deleted → re-login.
403	`NOT_APPROVED`	Dashboard accessed before approval → show waiting screen.
403	`ALREADY_APPROVED`	Editing onboarding after approval → route to profile.
400	`TYPE_REQUIRED` / `CONTENT_REQUIRED`	Document upload missing type/content.
422	`VALIDATION_FAILED`	Field/doc validation → render per-field `errors[]`.
500	`SERVER_ERROR`	Unexpected → generic retry message.

19 Authentication Tokens

Token	kind	Default expiry (env)	Used at
Access	`app`	12h (`APP_ACCESS_TOKEN_EXPIRY`)	Every protected endpoint (`Authorization: Bearer`).
Refresh	`app-refresh`	30d (`APP_REFRESH_TOKEN_EXPIRY`)	`POST /auth/refresh` only.

Claims

```
// access
{ "tempId": 123, "kind": "app", "iat", "exp" }
// after G1 (employee login) an employee-kind token is recommended:
{ "employeeId": 456, "kind": "app-employee", "iat", "exp" }
```

Client rules

- Store both tokens in secure storage (`flutter_secure_storage`) — never in plain prefs.
- Attach the access token to every request via a Dio interceptor.
- On `401 TOKEN_INVALID` : call `/auth/refresh` once; on success retry the original request; on failure clear tokens and route to login.
- On logout, discard both tokens (server logout is stateless).
- Tokens are signed with the server `JWT_SECRET` — the app never decodes for trust, only for optional expiry hinting.

> **Never** hardcode `JWT_SECRET` , base URLs with credentials, or any admin token in the app. The app only ever holds an employee-scoped app token obtained through the OTP flow.

20 File Upload Specifications

Files are transmitted as **base64 data URLs** inside JSON (no multipart today). This matches how the onboarding controllers store documents on the record.

Aspect	Spec
Transport	JSON field `dataUrl` : `data:<mime>;base64,<payload>`
Photo	JPEG/PNG; stored on `photoUpload` .
Documents	JPEG/PNG/PDF; stored in the `documents` JSON under a storage key.
Envelope per doc	`{ name, dataUrl, uploadedAt }`
Recommended max size	≤ 2–3 MB per file (compress client-side before base64; base64 inflates ~33%).
Type keys	photo, aadhaar→aadhaarDoc, pan→panDoc, passbook/bank→bankProof, signature, degree, education→educationCert, experience→experienceLetter, other

> **Client guidance.** Resize/compress images to ~1080px long edge and JPEG quality ~70% before encoding. Show an upload progress state; retry idempotently (re-uploading the same type overwrites its key). For large PDFs consider asking backend to add a multipart endpoint later — the current base64 path is fine for IDs and photos.

21 API Versioning

- The mobile API is namespaced at `/api/app` and advertises `version: "v1"` at the base path (`GET /api/app`).
- **Recommendation (Phase 15):** move to an explicit path version `/api/app/v1/...` before public release, so a future `/v2` can coexist. Until then, treat `/api/app` as v1.
- The app should send an `X-App-Version` header (build number) so the backend can, in future, enforce minimum-supported-version and return a "please update" response.

> **Compatibility contract.** Within v1, fields are only ever **added**, never removed or retyped. The app must ignore unknown fields and tolerate nulls (all read endpoints already return empty collections rather than erroring when a dataset is missing).

22 Security Guidelines

1. **Transport:** HTTPS only. Reject cleartext. Consider certificate pinning for production.
2. **Token storage:** secure keystore/keychain via `flutter_secure_storage`.
3. **Employees only:** app tokens are keyed to employee identities; `User`-backed admin roles can never obtain one. Keep it that way when implementing G1 (employee login) — resolve only against `Employee` / `TemporaryEmployee`.
4. **Ownership on every write:** the server derives the employee id from the token, never from the request body. For id-addressed actions (cancel leave, view loan/payslip), the server re-checks `row.employeeId === token employee`.
5. **Tenant isolation:** all reads are filtered by employee/company/branch server-side (§23–24).
6. **OTP throttling:** 5 attempts + 5-minute TTL already enforced. Add per-mobile rate limiting at the gateway for production.
7. **Audit:** writes flow through Prisma and the global audit middleware, so create/update actions are recorded.
8. **PII:** Aadhaar/PAN/bank are sensitive — mask in the UI where possible and never log payloads.

23 Company Isolation Rules

The platform is multi-company. An employee must never see another company's data.

- The logged-in identity carries its own `companyId` (on the `Employee` / `TemporaryEmployee` record).
- Every dashboard read is scoped by `employeeId = token employee`, which is itself inside one company — so records are inherently company-isolated.
- Cross-company reference reads (company name, holidays, notifications) are filtered by the identity's `companyId` — e.g. notifications: `WHERE companyId = e.companyId`; holidays (when built): `WHERE companyId = e.companyId`.
- No endpoint accepts a client-supplied `companyId`. The app cannot request another company's data even by tampering.

> **Verified.** The audited controllers derive `companyId` exclusively from the server-side identity. There is no code path where the app can widen its own company scope.

24 Branch Isolation Rules

- The identity carries `branchId`. Branch-scoped reads (notifications, and holidays/announcements when filtered) narrow to the employee's branch.
- Employee-owned data (attendance, leave, payroll, loans) is already the narrowest scope — a single employee — so it is branch-correct by construction.
- When building the holiday endpoint (G4), respect `CommunicationHoliday.applicableBranches` (a JSON array of branch ids or `"all"`) so branch-specific holidays don't leak across branches.

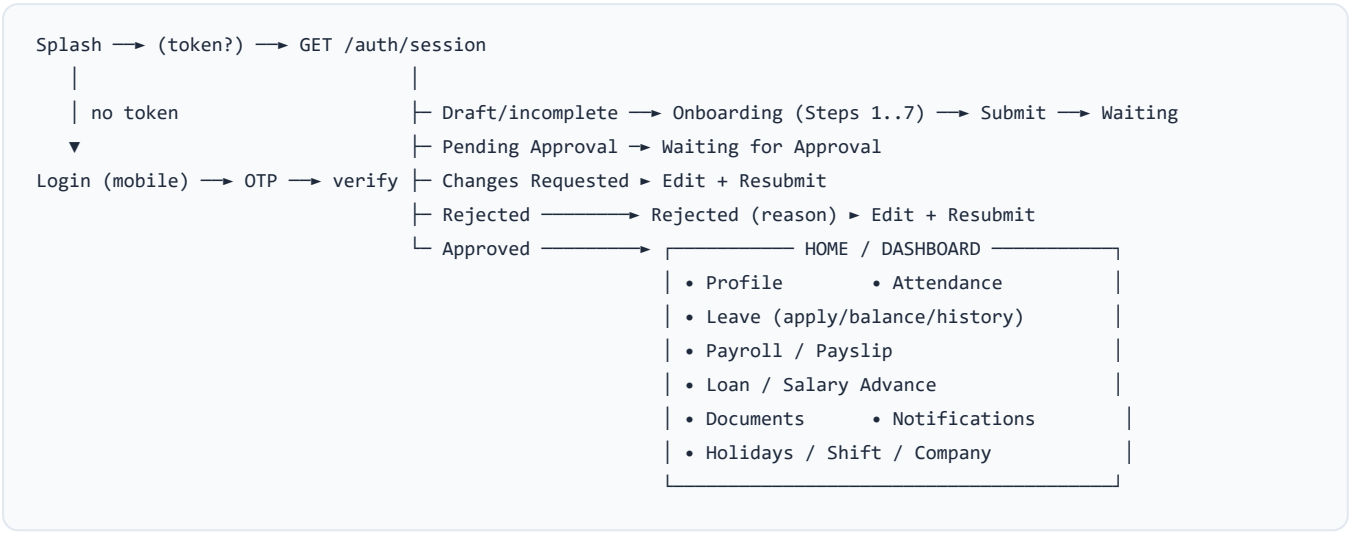
> **Isolation hierarchy:** `Company → Branch → Employee`. Every app response is confined to this chain, in that order, and always derived from the token — never the request.

25 Employee Permissions

The app exposes a fixed, employee-only capability set. There is no role matrix inside the app — an employee can only act on their own records.

Capability	Allowed?	Constraint
View own profile / dashboard	**Yes**	Own record only.
Edit onboarding draft	**Yes**	Until approval.
Limited self-service profile edit (post-approval)	**Yes**	`email, phone, addresses, emergencyContact` only.
View own attendance / payroll / leave / loan	**Yes**	Read-only, own records.
Apply leave / loan / advance	**Yes**	Creates a Pending request for HR approval.
Cancel own *pending* leave	**Yes**	Only `status="Pending"` AND owned.
Edit attendance / payroll / approve anything	**No**	Website-only.
See other employees' data	**No**	Structurally impossible.
Any admin/HR action	**No**	Not an app capability.

26 Complete Screen Flow



Screen → endpoint map

Screen	Primary endpoint(s)
Login / OTP	`/auth/login`, `/auth/verify-otp`
Onboarding wizard	`/profile/personal…documents`, `/profile/progress`, `/profile/submit`
Waiting / Rejected	`/profile/status`
Home	`/dashboard`
Profile / Company	`/profile`, `/company` *
Attendance	`/attendance`, `/attendance/monthly` *, `/overtime` *, `/shift` *
Leave	`/leave`, `/leave/apply`, `/leave/balance` *, `/leave/:id` *
Payroll / Payslip	`/payroll`, `/payroll/:id/payslip` *
Loan / Advance	`/loans` *, `/advances` *
Documents	`/profile/documents`, `/documents` *
Notifications	`/notifications`, `/notifications/:id/read` *
Holidays	`/holiday` *

* = endpoint to build (reuses an existing table; see Appendix B).

27 Database Relationships (High Level)

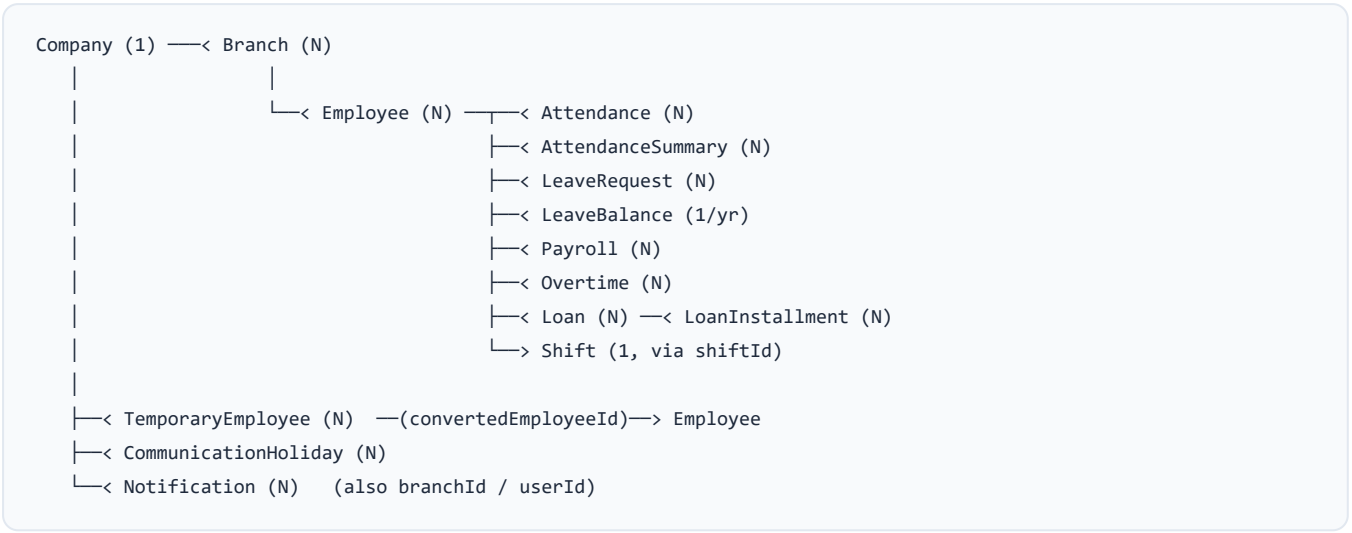
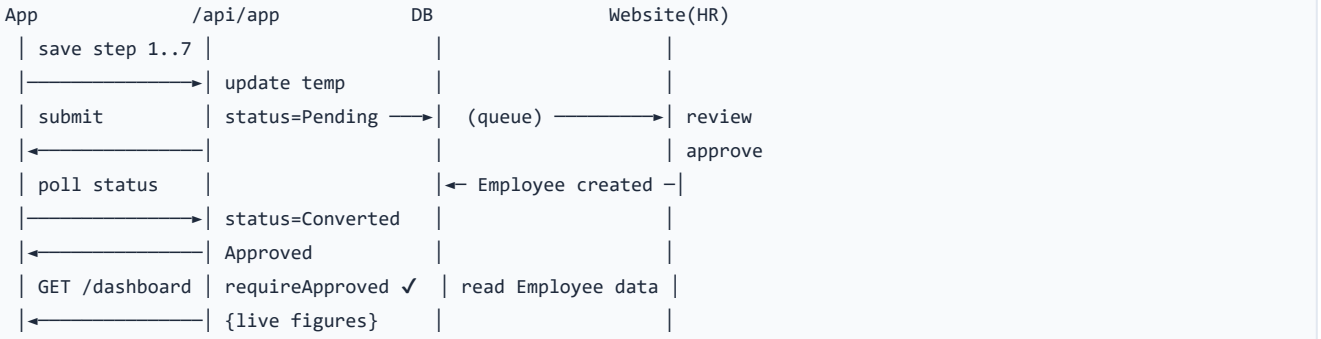


Table	Key columns the app uses
`Employee`	employeeId, companyId, branchId, shiftId, name, photoUpload, salary, pan, aadhaar, pfNumber, esiNumber, uan, bank*
`TemporaryEmployee`	mobile, status, selfProfile, documents, convertedEmployeeId, profileCompletion
`Attendance`	employeeId, date, status, clockIn/Out, hoursWorked
`AttendanceSummary`	workingDays, presentDays, weeklyOff, holidayDays, payableDays
`LeaveRequest` / `LeaveBalance`	leaveType, from/to, days, status / cl,pl,sl balance+used
`Payroll`	month, year, basicSalary, allowances, deductions, netSalary, paymentStatus, payslipGenerated
`Loan` / `LoanType`	loanNumber, principalAmount, emiAmount, tenureMonths, status, purpose
`CommunicationHoliday`	name, date, category, applicableBranches
`Notification`	type, title, message, read, companyId, branchId

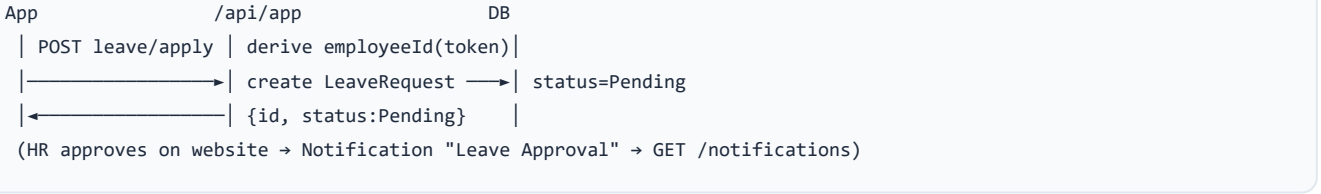
28 Sequence Diagrams

Login & token issue

Onboarding → Approval → Dashboard



Apply leave



29 Integration Checklist

Backend (before app GA)

- ☐ **G1** — add `Employee.phone` fallback to login + `app-employee` token path.
- ☐ **G4** — implement `/holiday` from `CommunicationHoliday` (currently a stub).
- ☐ **G2/G3** — `/leave/balance`, `DELETE /leave/:id` (ownership-guarded).
- ☐ **G5/G6** — `/shift`, `/overtime`.
- ☐ **G7/G8** — `/loans`, `/advances` (GET+POST, reuse Loan engine).
- ☐ **G10** — notifications mark-read.
- ☐ **G11** — post-approval document upload.
- ☐ **G12** — payslip download reusing the website renderer.
- ☐ **G13** — enrich `/dashboard` (photo, code, monthly attendance, leave balance, holidays, shift, CTC, PF/ESIC, loan/advance summary).
- ☐ Pagination/search/sort params on list endpoints.
- ☐ Set `OTP_MODE=production` + wire SMS provider when ready.

Flutter

- ☐ Dio client with base URL + auth interceptor + refresh-on-401.
- ☐ Secure token storage.
- ☐ Envelope model (`success/message/data/errors`) + typed error-code handling (§18).
- ☐ Splash router keyed on `/auth/session` + `approvalStatus`.
- ☐ Onboarding wizard driven by `/profile/progress` (server is the source of step state).
- ☐ Image compression before base64 upload (§20).
- ☐ No local salary/attendance/leave calculations — render server values only.
- ☐ Push notifications (future) — poll `/notifications` until FCM is added.

30 Flutter Implementation Notes

Response envelope model

```
class ApiResponse<T> {
  final bool success; final String message;
  final T? data; final List<ApiError> errors;
  ApiResponse.fromJson(Map j, T Function(dynamic) parse)
    : success = j['success'] ?? false,
      message = j['message'] ?? '',
      data = j['data'] == null ? null : parse(j['data']),
      errors = (j['errors'] ?? []).map((e)=>ApiError.fromJson(e)).toList();
}
class ApiError { final String code, message; final String? field; }
```

Auth interceptor (Dio)

```
dio.interceptors.add(InterceptorsWrapper(
  onRequest: (o, h) { o.headers['Authorization'] = 'Bearer $accessToken'; h.next(o); },
  onError: (e, h) async {
    if (e.response?.statusCode == 401) {
      final ok = await tryRefresh();          // POST /auth/refresh
      if (ok) return h.resolve(await retry(e.requestOptions));
      logoutToLogin();
    }
    h.next(e);
  },
));
```

Splash routing

```
final s = await api.session();          // GET /auth/session
switch (s.approvalStatus) {
  'Approved'          => go(Dashboard),
  'Pending Approval'  => go(WaitingApproval),
  'Rejected'          => go(Rejected),
  'Changes Requested' => go(EditResubmit),
  _                    => go(Onboarding, step: s.currentStep),
}
```

Golden rules

- **Server is truth.** Never compute salary, net pay, attendance totals, leave balances, or EMIs in the app.
- **State from server.** The onboarding `currentStep` / `completionPercentage` come from the server — don't keep a divergent local copy.
- **Tolerate nulls.** Read endpoints return empty collections when data is missing; render empty states, don't crash.
- **One base URL, one envelope, one error map.** Centralize all three.

Appendix A — Full Endpoint Audit (Phase 1)

Every mobile endpoint verified against source. **Auth:** app = token required; +A = approved (converted) employee required.

Endpoint	Method	Auth	Status	Isolation
`/api/app/`	GET	—	LIVE	n/a (health)
`/auth/login`	POST	—	LIVE	mobile lookup
`/auth/verify-otp`	POST	—	LIVE	session
`/auth/refresh`	POST	—	LIVE	refresh token
`/auth/logout`	POST	app	LIVE	self
`/auth/session`	GET	app	LIVE	self
`/auth/me`	GET	app	LIVE	self
`/profile/personal`	POST	app	LIVE	self
`/profile/address`	POST	app	LIVE	self
`/profile/family`	POST	app	LIVE	self
`/profile/bank`	POST	app	LIVE	self
`/profile/education`	POST	app	LIVE	self
`/profile/experience`	POST	app	LIVE	self
`/profile/documents`	POST	app	LIVE	self
`/profile/document`	POST	app	LIVE	self
`/profile/progress`	GET	app	LIVE	self
`/profile/submit`	POST	app	LIVE	self
`/profile/status`	GET	app	LIVE	self
`/profile/update`	PUT	app	LIVE	self
`/dashboard`	GET	app+A	LIVE (thin)	self+co+br
`/profile`	GET	app+A	LIVE	self+co+br
`/profile/documents`	GET	app+A	LIVE	self
`/attendance`	GET	app+A	LIVE (no paging)	self
`/leave`	GET	app+A	LIVE	self
`/leave/apply`	POST	app+A	LIVE	self+co
`/payroll`	GET	app+A	LIVE	self
`/notifications`	GET	app+A	LIVE	co+br
`/holiday`	GET	app+A	STUB ([])	—

Appendix B — Gap Analysis & Missing APIs (Phase 16/17)

Ordered by priority. Every item reuses an existing table — **no duplicate endpoints, no new business logic.**

ID	Gap	Fix (reuse)	Priority
G1	Login only checks `TemporaryEmployee`; permanent employees can't log in.	Add `Employee.phone` fallback + `app-employee` token.	**Blocking**
G4	`/holiday` returns `[]`.	`CommunicationHoliday` filtered by company/branch/upcoming.	**High**
G13	Dashboard missing most Phase-6 tiles.	Fold in AttendanceSummary, LeaveBalance, Shift, Payroll, Employee PF/ESIC, Loan summary.	**High**
G2	No leave balance.	`LeaveBalance` → `/leave/balance`.	**High**
G3	Cannot cancel pending leave.	`LeaveRequest` ownership-guarded delete.	**Medium**
G7	No loan apply/track in app.	`Loan` engine → `/loans` GET+POST.	**Medium**
G8	No salary advance.	`Loan` (advance type) → `/advances`.	**Medium**
G5	No shift details.	`Shift` via `Employee.shiftId`.	Medium
G6	No overtime view.	`Overtime` by employeeId.	Medium
G12	No payslip download.	Reuse website slip renderer.	Medium
G11	No post-approval doc upload.	`Employee.documents` / `Document`.	Low
G10	No notification mark-read.	`Notification` update.	Low
G9	No company info screen source.	`Company` + `Branch` → `/company`.	Low
—	No pagination/search/sort/versioned path.	Add params; move to `/api/app/v1`.	Low

Testing summary (Phase 17)

Category	Result
Working endpoints	**26 / 28 fully functional**
Thin/partial	**2 (dashboard, attendance — need enrichment/paging)**
Stub	**1 (holiday)**
Broken	**0**
Duplicate endpoints	**0 (isolated `/api/app` namespace)**
Security / isolation issues	**0 (all reads token-scoped; no client-supplied ids)**
Blocking gap	**1 (G1 — employee login)**

> **Bottom line.** The employee mobile backend is ~75% complete and architecturally sound: isolated namespace, clean envelope, real tenant isolation, and a working onboarding→approval→dashboard spine. Closing G1 unblocks all employees; the remaining gaps are additive, reuse-only endpoints that a developer can implement quickly against existing tables.